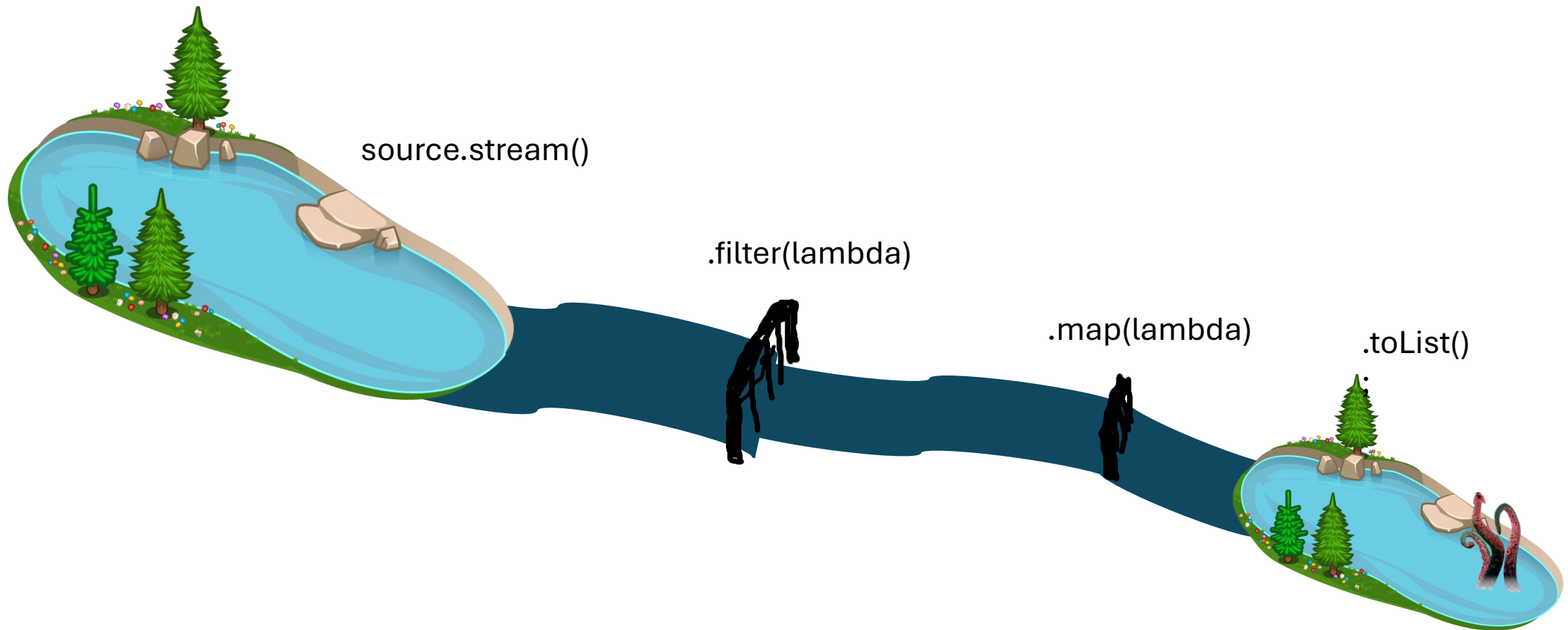


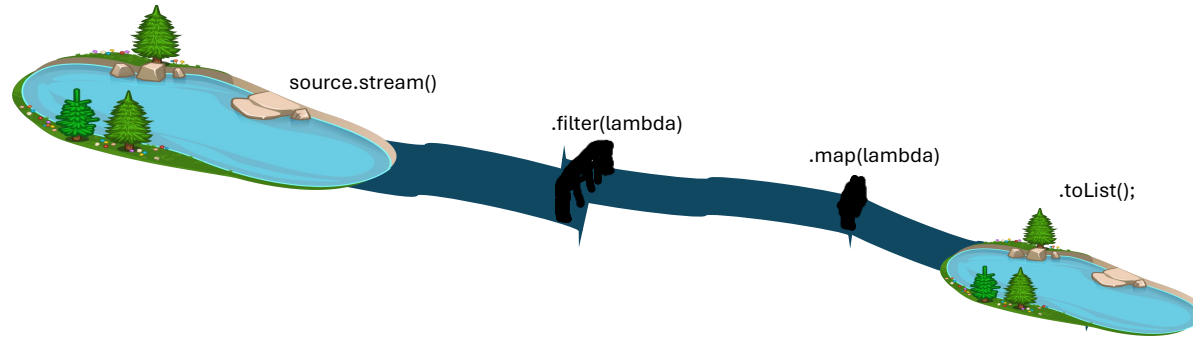
Stream

Stream

Das Interface `Stream<T>` aus `java.util.stream` beschreibt einen „Strom“ von Elementen. Dieser „fließt“ von einer Quelle zu einem Ziel. Auf seinem Weg kann der Strom nahezu beliebig manipuliert werden.



Stream



Stream-Erstellung

- collection.stream()
- Stream.of(...)
- ...

Intermediate Operatoren

- .filter(lambda)
- .map(lambda)
- .flatMap(lambda)
- .sorted(lambda)
- ...

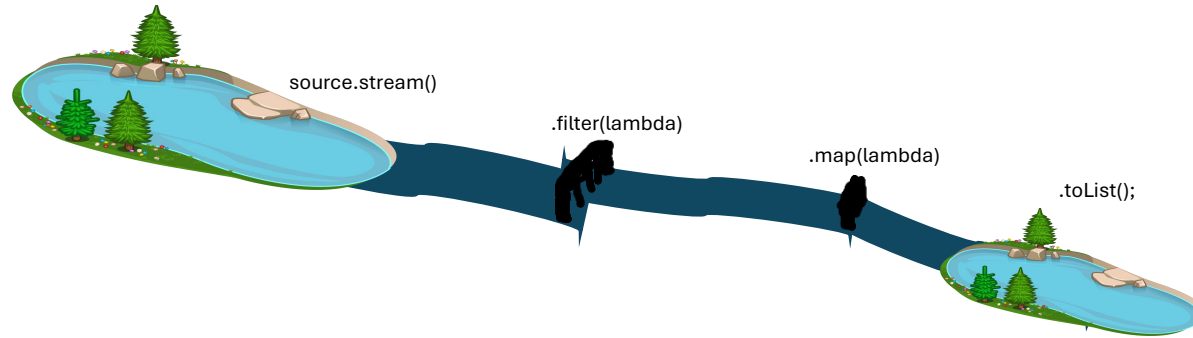
Terminale Operatoren

- .toList()
- .collect(...)
- .toList()
- .sum()
- .reduce()
- ...



```
Stream.of(3, 2, 1).filter(i -> i>1).sorted().toList();
```

Stream



Stream-Erstellung

- collection.stream()
- Stream.of(...)
- ...

Intermediate Operatoren

- .filter(lambda)
- .map(lambda)
- .flatMap(lambda)
- .sorted(lambda)
- ...

Terminale Operatoren

- .toList()
- .collect(...)
- .toList()
- .sum()
- .reduce()
- ...



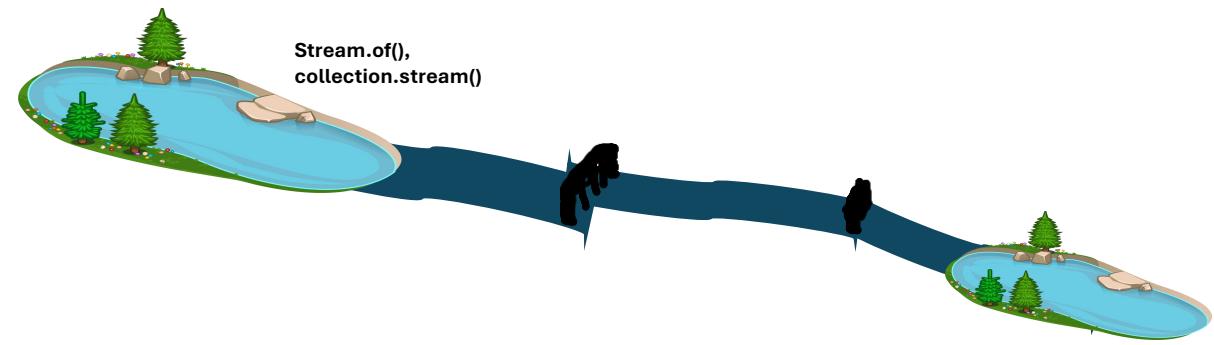
```
Stream.of(3, 2, 1).filter(i -> i>1).sorted().toList();
```



toList() gibt eine nicht veränderbare (immutable) List zurück.

D. h. die Ergebnisliste kann nicht verändert werden z. B. durch add, remove, set (UnsupportedOperationException)!

Stream erzeugen



```
Stream<Integer> numbers1 = Stream.of(1, 2, 3); // intern wird eine temporäre Liste erzeugt
```

```
Stream<Integer> numbers2 = List.of(1, 2, 3).stream();
```

```
Stream<Integer> numbers3 = new ArrayList<>(List.of(1, 2, 3)).stream();
```

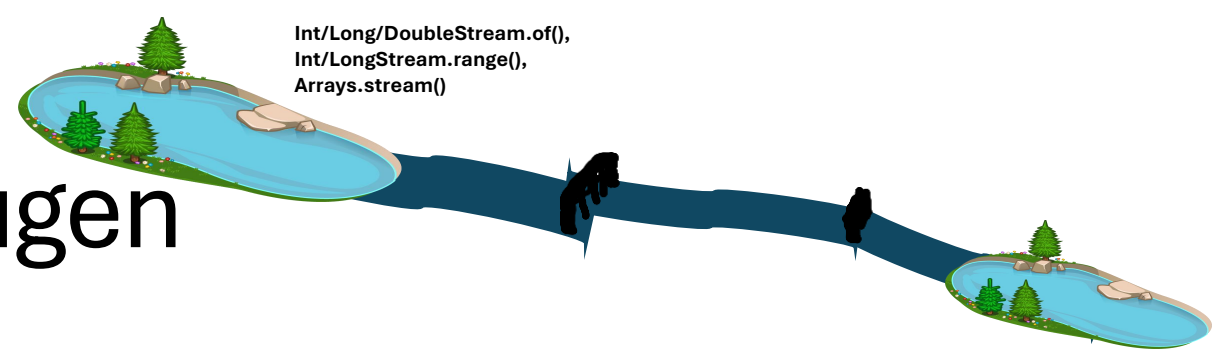
```
Stream<String> names1 = Stream.of("Bill", "Mark", "Jeff"); // temporäre Liste intern
```

```
Stream<String> names2 = List.of("Bill", "Mark", "Jeff").stream();
```

```
Stream<String> names3 = new ArrayList<>(List.of("Bill", "Mark", "Jeff")).stream();
```

🤖 Ein Stream selbst **speichert keine Elemente**, sondern greift auf eine vorhandene Datenquelle zu.
Ein Stream verarbeitet ausschließlich **Objekte** (d. h., auch **null** ist möglich).

Primitiven Stream erzeugen



Int/Long/DoubleStream.of(),
Int/LongStream.range(),
Arrays.stream()

IntStream, LongStream und DoubleStream sind spezielle Streams, die **keine Objekte** sondern **primitive Daten** speichern (kein `null` möglich). Diese Streams können **effizienter und performanter** sein, sobald sehr viele Elemente vorhanden sind (da keine Objekte verwaltet werden müssen, d. h., weniger Overhead).

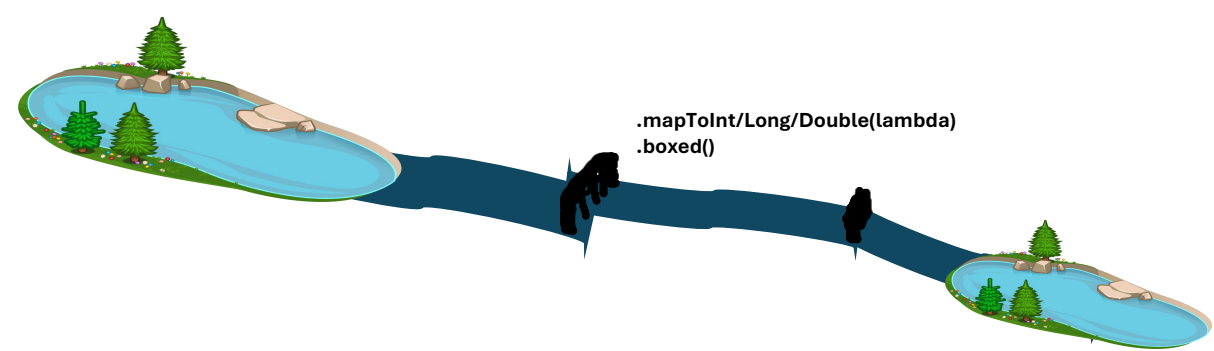
```
IntStream    ints1    = IntStream.of(1, 2, 3);
IntStream    ints2    = IntStream.range(1, 4);    // gleich wie ints1
LongStream   longs1   = LongStream.of(1, 2, 3);
LongStream   longs2   = LongStream.range(1, 4);   // gleich wie longs1
DoubleStream doubles  = DoubleStream.of(1, 2, 3); // kein range möglich
```

Alternativ kann man primitive Streams mit einem vorhandenen Array über `Arrays.stream(array)` erzeugen:

```
IntStream    ints3    = Arrays.stream(new int[] {1, 2, 3});
LongStream   longs3   = Arrays.stream(new long[] {1, 2, 3});
DoubleStream doubles2 = Arrays.stream(new double[] {1, 2, 3});
```

Weiterhin verfügen primitive Streams über **zusätzliche oder spezialisierte Methoden** wie z. B. `min()`, `max()`, `average()`, `sum()`, `mapToInt/Long/Double/Obj()`.

Streams umwandeln



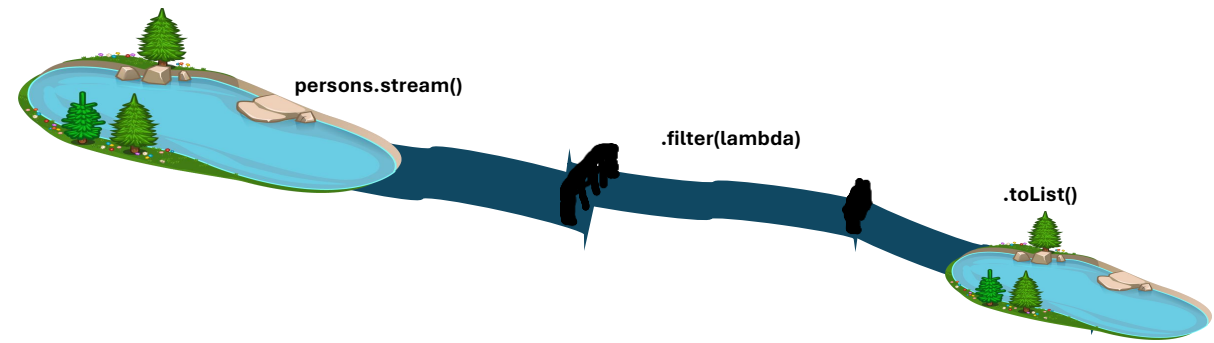
Um einen primitiven Stream **in einem Objekt-Stream umzuwandeln**, kann **mapToObj()** oder **boxed()** verwendet werden (jedes primitive Element wird in ein entsprechendes Objekt gepackt).

```
Stream<Integer> intObjects    = IntStream.of(1, 2, 3).boxed();  
Stream<Long>     longObjects   = LongStream.of(1, 2, 3).boxed();  
Stream<Double>   doubleObjects = DoubleStream.of(1, 2, 3).boxed();
```

Ein Objekt-Stream kann mithilfe von `mapToInt/Long/Double(lambda)` in einen primitiven Stream umgewandelt werden.

```
IntStream      ints          = Stream.of(1l, 2l, 3l).mapToInt(l -> l.intValue());  
LongStream     longs         = Stream.of(1d, 2d, 3d).mapToLong(d -> d.longValue());  
DoubleStream   doubles       = Stream.of(1, 2, 3).mapToDouble(i -> i);
```

Stream – Beispiel 1



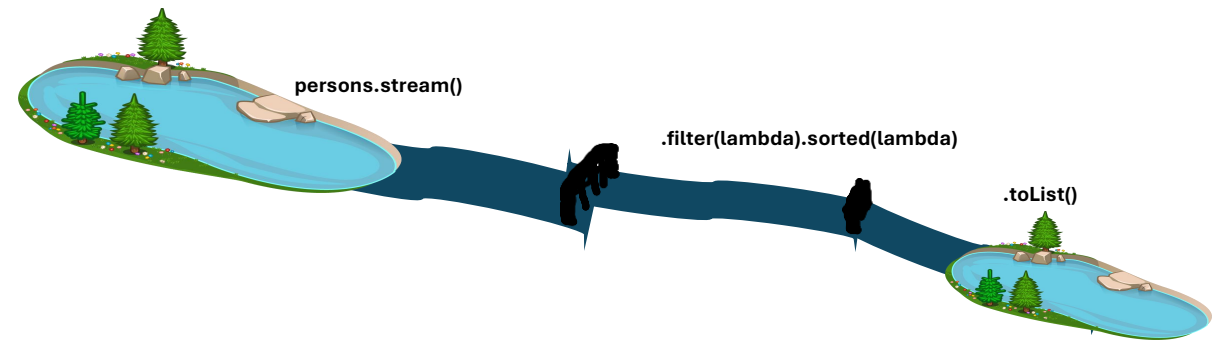
```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
  
        List<Person> minors = persons.stream()  
            .filter(person -> person.age() < 18)  
            .toList();  
        IO.println(minors);  
    }  
}
```

Ausgabe:

```
[Person[name=Bob, age=10],  
Person[name=Alice, age=15]]
```

person.age() ist ein Getter aus dem record Person und gibt ein int zurück.

Stream – Beispiel 2



```
record Person(String name, int age) {
```

```
void main() {
```

```
List<Person> persons = List.of(
```

```
    new Person("Bob", 10),
```

```
    new Person("Alice", 15),
```

```
    new Person("Jane", 20)
```

```
);
```

```
List<Person> minors = persons.stream()
```

```
    .filter(person -> person.age() < 18)
```

```
    .sorted((person1, person2) -> person1.name().compareTo(person2.name()))
```

```
    .toList();
```

```
IO.println(minors);
```

```
}
```

```
}
```

Ausgabe:

```
[Person[name=Alice, age=15],  
Person[name=Bob, age=10]]
```

person1.name() ist ein Getter aus dem record Person und gibt einen String zurück.

compareTo ist eine Methode aus der Klasse String (die das Comparable-Interface implementiert) und gibt ein int zurück, der die Reihenfolge bestimmt.



Stream – Essentielles Wissen

- Ein Stream selbst **speichert keine Elemente**. Er bezieht sie aus einer **Quelle**. Typische Quellen sind Collections (z. B. ArrayList, HashSet, Arrays), aber auch andere Quellen sind möglich.
- Nach der Erzeugung eines Streams, können **beliebig** viele **intermediate Operationen** zum Stream hinzugefügt werden.
- Ein Stream ist im Prinzip eine **Sammlung von auszuführenden Funktionen (aka Lambdas)**.
- Streams sind **lazy**. Die enthaltenen **Funktionen** werden erst **durch Aufruf einer terminalen Operation nacheinander ausgeführt**.
- Ein Stream wird durch eine terminale Operation geschlossen. Der Stream kann anschließend **nicht wiederverwendet** werden!

filter(lambda)

intermediate operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );
```

// for-each-Variante-----

```
List<Person> minors = new ArrayList<Person>();  
for (Person person : persons)  
    if (person.age() < 18)  
        minors.add(person);  
IO.println(minors);
```

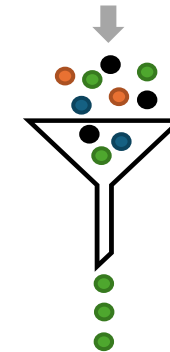
// Stream-----

```
minors = persons.stream()  
    .filter(person -> person.age() < 18)  
    .toList();  
IO.println(minors);  
}  
}
```

Merkhilfe:

to filter: etwas ausfiltern

„Gib mir nur die Elemente zurück,
die ich behalten will.“



Ausgabe:

```
[Person[name=Bob, age=10], Person[name=Alice, age=15]]  
[Person[name=Bob, age=10], Person[name=Alice, age=15]]
```

map(lambda)

intermediate operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );
```

// for-each-Variante -----

```
List<String> modifiedNames = new ArrayList<String>();  
for (Person person : persons) {  
    String modifiedName = person.name().toUpperCase();  
    modifiedNames.add(modifiedName);  
}  
IO.println(modifiedNames);
```

// Stream -----

```
modifiedNames = persons.stream()  
    .map(person -> person.name().toUpperCase()) // Es wird im Lambda von map ein String zurückgegeben, deswegen  
    .toList(); // enthält das Ergebnis der map-Methode eine Liste von Strings  
IO.println(modifiedNames);  
}
```

Merkhilfe:

to map: etwas abbilden

„Ich mappe mir die Welt, wie sie mir gefällt!“
Aus Objekt Person("Bob", 10) wird String "BOB"

Ausgabe:

```
[BOB, ALICE, JANE]  
[BOB, ALICE, JANE]
```

flatMap(lambda)

intermediate operation

```
record Person(String name, int age) {  
    main() {  
        List<List<Person>> teams = List.of(  
            List.of(new Person("Bob", 10), new Person("Alice", 15)), // Sublist1  
            List.of(new Person("Jane", 20), new Person("Anna", 17)) // Sublist2  
        );
```

// for-each-Variante-----

```
List<Person> persons = new ArrayList<>();  
for (List<Person> team : teams) {  
    for (Person person : team) { // Alternativ: persons.addAll(team);  
        persons.add(person);  
    }  
}  
IO.println(persons);
```

// Stream -----

```
persons = teams.stream().flatMap(person -> person.stream()).toList();  
// persons = teams.stream().flatMap(List::stream).toList();  
IO.println(persons);  
}
```

Hinweise:

- Stream::flatMap erwartet, dass das Lambda einen Stream zurückgibt.
- Flatmap löst nur eine Verschachtelungs-Tiefe auf: Falls Sublist wiederum Listen enthalten würde, würden diese Subsublist in der Ergebnisliste landen.

Merkhilfe:

to flatMap: "flachklopfen"

Mache aus einer "Liste von Listen" eine Liste.

Beispiel:

$[[1, 2, 3], [4, 5], [6, [7], 8]] \rightarrow [1, 2, 3, 4, 5, 6, [7], 8]$.

Ausgabe:

```
[Person[name=Bob, age=10],  
Person[name=Alice, age=15],  
Person[name=Jane, age=20],  
Person[name=Anna, age=17]]
```

```
[Person[name=Bob, age=10],  
Person[name=Alice, age=15],  
Person[name=Jane, age=20],  
Person[name=Anna, age=17]]
```

mapToInt/Long/Double(lambda)

intermediate operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
    }  
}
```

// for-each-Variante -----

```
int sumAge = 0;  
for (Person person : persons)  
    sumAge = sumAge + person.age();  
IO.println(sumAge);
```

// Stream -----

```
sumAge = persons.stream()  
    .mapToInt(person -> person.age()) // mapToInt ist eine spezielle Variante von map, die Streams mit Primitiven erzeugt.  
    // .mapToInt(Person::age())        // Alternative Schreibweise mittels Methodenreferenz.  
    .sum();                             // mapToInt liefert einen IntStream zurück, der u. a. die sum()-Methode enthält.  
IO.println(sumAge);  
}
```

Merkhilfe:

mapToInt liefert ein IntStream mit Primitiven zurück, der zusätzliche Methoden enthält, z. B. min, max, sum und average.

Ausgabe:

45
45

IntStream.range(lambda) und indexOf

intermediate operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 2),  
            new Person("Alice", 3),  
            new Person("Jane", 1));
```

```
// for-each-Variante -----  
int indexOfAlice = -1;  
for (int i = 0; i < persons.size(); i++) {  
    Person person = persons.get(i);  
    if (person.name().equals("Alice")) {  
        indexOfAlice = i;  
        break; // breche die Schleife ab und gebe Ergebnis sofort zurück (short-circuiting).  
    }  
}  
IO.println(indexOfAlice);
```

// Statt die for-each-Variante ist oftmals indexOf aus dem List-Interface einfacher:

```
indexOfAlice = persons.indexOf(new Person("Alice", 3)); // Achtung: indexOf verwendet intern die equals-Methode von Person, um das passende Objekt zu finden.  
// Da Person ein record ist, wurde automatisch equals generiert, das name und age vergleicht.
```

```
// Stream -----  
indexOfAlice = IntStream.range(0, persons.size()) // Erzeugt einen IntStream mit allen möglichen Indices für persons.  
    .filter(i -> persons.get(i).name.equals("Alice")) // Finde alle passenden Objekte (filter verwendet intern kein break). Das Ergebnis ist eine Liste von gefilterten Indices.  
    .findFirst() // Nimm den ersten Index aus der gefilterten Liste.  
    .orElse(-1); // Falls es bei filter kein Match gab, gibt es keine Indices in der gefilterten Liste. Es wird dann -1 zurückgegeben.  
IO.println(indexOfAlice);  
}  
}
```

Dies ist zurzeit die "eleganteste" Lösung, um mit Streams den Index von einem gesuchten Objekt zu finden.

Ausgabe:

1
1
1

sorted(lambda)

intermediate operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 2),  
            new Person("Alice", 3),  
            new Person("Jane", 1));
```

```
// for-each-Variante -----  
List<Person> sortedPersons = new LinkedList<Person>(persons);  
sortedPersons.sort((p1, p2) -> Integer.compare(p1.age(), p2.age()));  
// Collections.sort(sortedPersons, (p1, p2) -> Integer.compare(p1.age(), p2.age()));  
IO.println(sortedPersons);
```

```
// Stream -----  
sortedPersons = persons.stream()  
    .sorted((p1, p2) -> Integer.compare(p1.age(), p2.age()))  
    // .sorted(Comparator.comparingInt(Person::age)) // Alternative Schreibweise.  
    .toList();  
IO.println(sortedPersons);
```

```
}
```

Merke:

sorted erhält zwei Parameter, die miteinander verglichen werden und benötigt ein int als Rückgabe für die Reihenfolge.

Ausgabe:

```
[Person[name=Jane, age=1],  
Person[name=Bob, age=2],  
Person[name=Alice, age=3]]
```

```
[Person[name=Jane, age=1],  
Person[name=Bob, age=2],  
Person[name=Alice, age=3]]
```

Hinweis: sorted erwartet ein Comparator<T>-Interface, das zwei Parameter entgegennimmt und ein int-Rückgabewert hat.

Die Reihenfolge wird durch das Vorzeichen des int-Rückgabewertes bestimmt: negativ (A kleiner B), 0 (A gleich B) oder positiv (A größer B).

skip/limit(lambda)

intermediate operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 2), // skipped  
            new Person("Alice", 3),  
            new Person("Jane", 1),  
            new Person("Bill", 4) // limited  
        );  
  
        // for-each-Variante -----  
        List<Person> result = new ArrayList<>();  
        int skip = 1;  
        int limit = 2;  
        for (Person person : persons) {  
            if (skip-- > 0)  
                continue;  
            if (limit-- == 0)  
                break;  
            result.add(person);  
        }  
        IO.println(result);  
  
        // Stream -----  
        result = persons.stream()  
            .skip(1) // überspringe die ersten x Elemente.  
            .limit(2) // gebe maximal y Elemente zurück.  
            .toList();  
        IO.println(result);  
    }  
}
```

Merke:

*Überspringe die ersten x Elemente (skip)
oder gebe maximal x Elemente (limit)
zurück.*

Ausgabe:

```
[Person[name=Alice, age=3], Person[name=Jane, age=1]]  
[Person[name=Alice, age=3], Person[name=Jane, age=1]]
```

toList(lambda)

terminal operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
  
        // Stream -----  
        List<Person> newPersons = persons.stream().toList(); // nicht modifizierbar  
        newPersons.add(new Person("Bill", 10)); // wirft Exception zur Laufzeit  
    }  
}
```

Merkhilfe:

Gibt eine nicht-modifizierbare Liste zurück. D. h., add-, remove-, set-Methoden werfen zur Laufzeit eine UnsupportedOperationException.

Ausgabe:

UnsupportedOperationException

collect(lambda)

terminal operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
  
        // Stream -----  
        List<Person> newPersons = persons.stream().collect(Collectors.toList());  
        newPersons.add(new Person("Bill", 2)); // eine modifizierbare Liste  
  
        List<Person> newPersonsSet = persons.stream().collect(Collectors.toSet());  
        newPersonsSet.add(new Person("Bob", 10)); // ein modifizierbares Set  
    }  
}
```

Merkhilfe:

Ermöglicht es, die Rückgabe einer Streams beliebig anzupassen.

Die Klasse [Collectors](#) enthält hierzu viele Hilfsmethoden.

forEach(lambda)

terminal operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
    }  
}
```

// for-each-Variante-----

```
for (Person person : persons)  
    IO.println(person.name());
```

// Stream -----

```
persons.forEach(person ->  
    IO.println(person.name()));  
}  
}
```

Achtung: Verwechsle forEach nicht mit map!

forEach gibt keine Objekte zurück (void) und ist eine terminale Methode, während map eine Liste mit modifizierten Objekten zurückgibt und eine intermediate Operation ist.

Merkhilfe:

Alternative zur for-Schleife.
forEach ist eine terminale Methode.
Verwechsel forEach nicht mit map:
map ist intermediate und gibt
modifizierte Werte zurück. forEach
hat keinen Rückgabewert (void)!

Ausgabe:

Bob
Alice
Jane

Bob
Alice
Jane

reduce(lambda)

terminal operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );  
    }  
}
```

// for-each-Variante -----

```
int sumAge = 0;  
for (Person person : persons)  
    sumAge = sumAge + person.age;  
IO.println(sumAge);
```

// Stream -----

```
sumAge = persons.stream()  
    .map(person -> person.age) // map wandelt die Liste von Personen in eine Liste von Zahlen (age) um.  
    .reduce(0, (subtotal, age) -> subtotal + age); // subtotal wird mit 0 initialisiert. age ist jede Zahl aus der map-Methode.  
IO.println(sumAge);  
}  
}
```

Merkhilfe:

*to reduce: etwas verkleinern
„Reduziere die Liste zu einem
einzigem Wert.“*

[1 2 3 4 5] → 15

Ausgabe:

45

45

any/all/noneMatch(lambda)

terminal operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 10),  
            new Person("Alice", 15),  
            new Person("Jane", 20)  
        );
```

// for-each-Variante -----

```
boolean hasAnyPersonIceInName = false;  
for (Person person : persons) {  
    if (person.name().contains("ice")) {  
        hasAnyPersonIceInName = true;  
        break; // short-circuiting  
    }  
}  
IO.println(hasAnyPersonIceInName);
```

// Stream -----

```
hasAnyPersonIceInName = persons.stream()  
    .anyMatch(person -> person.name().contains("ice"));  
IO.println(hasAnyPersonIceInName);  
}  
}
```

Merkhilfe:

to match: übereinstimmen

„Ist das vorhanden?“

→ true oder false

Ausgabe:

true

true

min/max(lambda)

terminal operation

```
record Person(String name, int age) {  
    void main() {  
        List<Person> persons = List.of(  
            new Person("Bob", 2),  
            new Person("Alice", 3),  
            new Person("Jane", 1));
```

// for-each-Variante -----

```
Person result = null;  
for (Person person : persons) {  
    if (result == null || person.age() < result.age()) {  
        result = person;  
    }  
}  
IO.println(result);
```

// Stream -----

```
Optional<Person> optionalResult = persons.stream()  
    .min((p1, p2) -> Integer.compare(p1.age(), p2.age()));  
// .min(Comparator.comparingInt(Person::age)); // Alternative Schreibweise.  
IO.println(optionalResult); // Achtung: min und max geben als Ergebnis ein Optional-Objekt zurück.  
}
```

Merke:

min/max erhält zwei Parameter, die miteinander verglichen werden und benötigt ein int als Rückgabe für die Reihenfolge, genau wie beim Comparator<T>-Interface:

- **negativ Zahl:** Element A kleiner als Element B
- **positive Zahl:** Element A größer als Element B
- **0:** Element A ist gleich wie Element B
(ursprüngliche Reihenfolge bleibt erhalten)

Ausgabe:

```
Person[name=Jane, age=1]  
Optional[Person[name=Jane, age=1]]
```

java.util.Optional<T>

Manche Stream-Methoden geben ein Optional<T> zurück.

Beispiel:

```
IO.println(IntStream.of(1, 2, 3).average());
```

Ausgabe: OptionalDouble[2.0]

Gibt ein Optional<T> (hier sogar ein spezielles OptionalDouble) zurück, da bei average() theoretisch eine "Division by Zero" möglich ist, falls der Stream leer ist.

```
IO.println(IntStream.of().average());
```

Ausgabe: OptionalDouble.empty

Hinweis:

Stream-Methoden, die ein Optional zurückgeben, sind zum Beispiel: findFirst, findAny, min, max, reduce, und Int/Long/DoubleStream::average

java.util.Optional<T>

Optional<T> kann null-Werte kapseln, um Null-Pointer-Exceptions zu vermeiden:

// Problem:

```
String greeting;  
IO.println(greeting.toLowerCase()); // Null-Pointer-Exception
```

// Lästig:

```
if (greeting != null) // überall immer wieder null-Checks  
    IO.println(greeting.toLowerCase());
```



java.util.Optional<T>.get()

Optional<T> kann null-Werte kapseln, um Null-Pointer-Exceptions zu vermeiden:

// Problem:

```
String greeting;  
IO.println(greeting.toLowerCase()); // Null-Pointer-Exception
```

// Lästig:

```
if (greeting != null) // überall immer wieder null-Checks  
—IO.println(greeting.toLowerCase());
```



// Mit Optional<T>:

```
import java.util.Optional;  
Optional<String> greeting = Optional.ofNullable(aVariableThatMightBeNull);
```

// Beispiel bad practice:

```
if (greeting.isPresent()) // kein Gewinn zu if(greeting != null)  
    IO.println(greeting.get());
```



java.util.Optional<T>.orElse(defaultValue)

Optional<T> kann null-Werte kapseln, um Null-Pointer-Exceptions zu vermeiden:

// Problem:

```
String greeting;  
IO.println(greeting.toLowerCase()); // Null-Pointer-Exception
```

// Lästig:

```
if (greeting != null) // überall immer wieder null-Checks  
—IO.println(greeting.toLowerCase());
```



// Mit Optional<T>:

```
import java.util.Optional;  
Optional<String> greeting = Optional.ofNullable(aVariableThatMightBeNull);
```

// Beispiel bad practice:

```
if (greeting.isPresent()) // kein Gewinn zu if(greeting != null)  
—IO.println(greeting.get());
```



// Beispiel good practice:

```
greeting.orElse("Hi"); // return Wert oder Default-Wert wenn null  
greeting.orElseThrow(() -> IllegalArgumentException::new) // return Wert oder Exception wenn null
```



Stream vs for-each (oldschool)

```
void main() {  
    var numbers = List.of(1, 2, 3, 5, 4, 7, 8, 9);
```

```
    // Variante 1: for-each (oldschool)
```

```
    int result = 0;  
    for (int e : numbers) {  
        if (e > 3 && e % 2 == 0) {  
            result = e * 2;  
            break;  
        }  
    }  
    IO.println(result);
```

```
    // Variante 2: Stream
```

```
    result = numbers.stream()  
        .filter(e -> e > 3)  
        .filter(e -> e % 2 == 0) // FYI: man könnte beide Filtermethoden auch mit && zusammenfassen.  
        .map(e -> e * 2)  
        .findFirst()  
        .orElse(0); // Keine Stream-Methode sondern aus der Klasse Optional  
    IO.println(result);  
}
```

Ausgabe:

8
8

Im Vergleich: Streams sind ausdrucksstärker und beschreiben mehr, was der Code macht. 😎

Stream – Intermediate Operations

Intermediate Operationen **geben immer `Stream<T>` zurück**. Eine vollständige Liste findest du [hier](#).

<code>Stream<T> filter(lambda)</code>	Liefert die Elemente zurück, die das vorgegebene Prädikat erfüllen.
<code>Stream<T> map(lambda)</code>	Mache etwas mit jedem Element und gebe die modifizierten Elemente zurück.
<code>Stream<T> flatMap(lambda)</code>	Listen "flachklopfen": <code>[[1, 2, 3], [4, 5], [6, [7], 8]] → [1, 2, 3, 4, 5, 6, [7], 8]</code> .
<code>Stream<T> sorted(lambda)</code>	Sortiere die Liste (siehe Interface Comparator<T>).
<code>Stream<T> distinct()</code>	Entferne alle Duplikate.
<code>Stream<T> limit(long maxSize)</code>	Gebe die ersten x Elemente zurück.
<code>Stream<T> skip(long n)</code>	Überspringe die ersten x Elemente.
<code>Stream<T> boxed()</code>	Wandelt einen primitiven Stream zu einem Objekt-Stream um.
<code>Stream<T> peek(lambda)</code>	Zum Debuggen, um sich Zwischenergebnisse eines Streams anzuschauen.

Stream – Terminal Operations

Terminal Operations geben unterschiedliche Werte zurück, je nach Methode. Eine vollständige Liste findest du [hier](#).

<code>List<T> toList()</code>	Gebe alle Elemente des Streams als <i>unmodifiable</i> List zurück.
<code>T[] toArray(T[]::new)</code>	Gebe alle Elemente des Streams als Array vom Typ T zurück.
<code>Collection<T> collect(lambda)</code>	Ermöglicht mutable reduction - und Collectors -Operationen (z. B. <code>teeing</code> , <code>joining</code> , <code>groupingBy</code>).
<code>T/Optional<T> reduce(lambda)</code>	Reduziert Elemente zu einem Element (z. B. Summe aller Zahlen).
<code>void forEach(lambda)</code>	Iteriere über jedes Element (wie <code>for</code> -Schleife). Kein Rückgabewert!!!
<code>long count()</code>	Gebe die Anzahl der Elemente zurück.
<code>boolean anyMatch(lambda)</code>	Gebe <code>true</code> zurück, wenn irgendein Element die gegebene Bedingung erfüllt.
<code>boolean allMatch(lambda)</code>	Gebe <code>true</code> zurück, wenn alle Elemente die gegebene Bedingung erfüllen.
<code>boolean noneMatch(lambda)</code>	Gebe <code>true</code> zurück, wenn kein Element die gegebene Bedingung erfüllt.
<code>Optional<T> findAny(lambda)</code>	Gibt ein zufälliges Element zurück (falls vorhanden).
<code>Optional<T> findFirst(lambda)</code>	Gibt das erste Element zurück (falls vorhanden).
<code>Optional<T> min(lambda)</code>	Gebe das kleinste Element anhand des gegebenen Comparator<T> zurück (falls vorhanden).
<code>Optional<T> max(lambda)</code>	Gebe das größte Element anhand des gegebenen Comparator<T> zurück (falls vorhanden).